

1 Theory

ALU

An Arithmetic Logic Unit (ALU) is a fundamental component in digital circuits and system design, responsible for performing arithmetic and logical operations in processors. It is a combinational circuit that integrates basic operations like addition, subtraction, bitwise operations (AND, OR, XOR, NOT) comparisons (equality, greater than, less than) and complements. Control signals determine the operation, and outputs include results and status flags such as carry, zero, negative, and overflow.

8-Bit ALU

An 8-bit ALU is a combinational circuit which operates on two 8-bit input buses depending on what the selection inputs are. 8-bit ALU performs common arithmetic and logical operations as mentioned above. These operations are common to all computer systems and thus are an integral part of computer. This ALU design is unique, effective, and minimally engineered.

2 Implementation and Observation

2.1 8-Bit ALU

VHDL code - main program

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_unsigned.all;

entity alu is
    Port ( a : in STD_LOGIC_VECTOR (7 downto 0);
          b : in STD_LOGIC_VECTOR (7 downto 0);
          alu_sel : in STD_LOGIC_VECTOR (3 downto 0);
          alu_out : out STD_LOGIC_VECTOR (7 downto 0);
          carry_out : out STD_LOGIC);
end alu;

architecture Behavioral of alu is
    signal temp1: std_logic_vector(8 downto 0);
begin

    process(a,b,alu_sel)
    begin
        case alu_sel is
            when "0000" => alu_out <= a;
```

```

when "0001" =>
temp1 <= ('0' & a) + ('0' & b);
alu_out <= temp1(7 downto 0);
carry_out <= temp1(8);
when "0010" => alu_out <= a-b;
when "0011" => alu_out <= a and b;
when "0100" => alu_out <= a or b;
when "0101" => alu_out <= a xor b;
when "0110" => alu_out <= a nand b;
when "0111" => alu_out <= a xnor b;
when "1000" => alu_out <= a(6 downto 0) & '0'; --shift left
when "1001" => alu_out <= '0' & a(6 downto 0); --shift right
when "1010" => alu_out <= a(6 downto 0) & a(7); --rotate left
when "1011" => alu_out <= a(7) & a(6 downto 0); --rotate right

when "1100" => --equal comparision
if(a = b) then
alu_out <=x"01";
else
alu_out <= x"00";
end if;

when "1101" => --greater than comparision
if(a > b) then
alu_out <=x"01";
else
alu_out <= x"00";
end if;

when "1110" => alu_out <= not a;
when "1111" => alu_out <= not a + '1';

when others => alu_out <= not a;

end case;
end process;

end Behavioral;

```

VHDL code - testbench file

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_unsigned.all;

```

```

entity alu_tb is
    --Port ();
end alu_tb;

architecture Behavioral of alu_tb is
    component alu is
        Port ( a : in STD_LOGIC_VECTOR (7 downto 0);
              b : in STD_LOGIC_VECTOR (7 downto 0);
              alu_sel : in STD_LOGIC_VECTOR (3 downto 0);
              alu_out : out STD_LOGIC_VECTOR (7 downto 0);
              carry_out : out STD_LOGIC);
    end component;

    signal a : STD_LOGIC_VECTOR (7 downto 0) := "00000000";
    signal b : STD_LOGIC_VECTOR (7 downto 0) := "00000000";
    signal alu_sel : STD_LOGIC_VECTOR (3 downto 0) := "0000";
    signal alu_out : STD_LOGIC_VECTOR (7 downto 0) := "00000000";
    signal carry_out : STD_LOGIC := '0';

begin
    uut: alu port map(a => a,
                     b => b,
                     alu_sel => alu_sel,
                     alu_out => alu_out,
                     carry_out => carry_out);

    process
    begin

        for i in 0 to 32 loop
            a <= "00000011";
            b <= "00000001";
            alu_sel <= alu_sel + "0001";
            wait for 10ns;
        end loop;

    end process;

end Behavioral;

```

RTL Schematic Diagram

RTL design is used to describe the behavior of the ALU as shown in figure 1

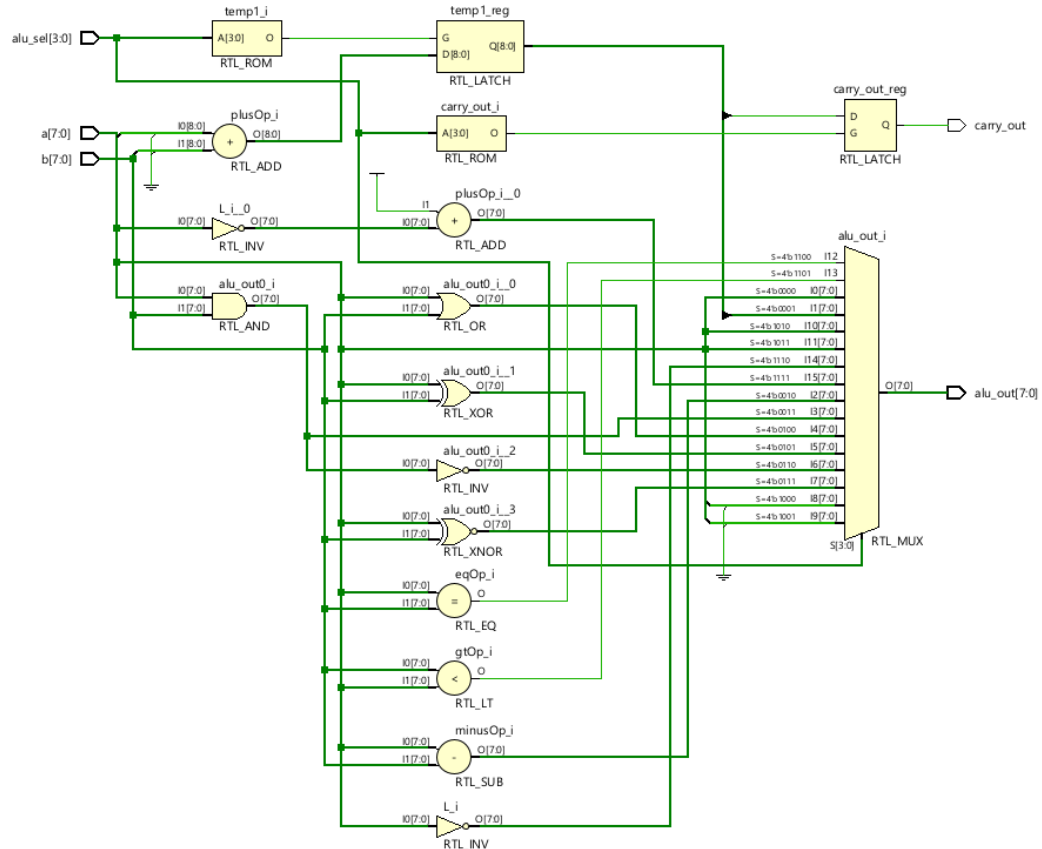


Figure 1: RTL Schematic Diagram of ALU

Synthesis Schematic Diagram

Schematic design is used to describe the implementation of the ALU circuit as shown in figure 2. It involves converting a VHDL code into a gate-level representation through synthesis, generating a netlist that optimizes for factors like speed and power. The resulting schematic visually represents the gates and interconnections, allowing engineers to verify and optimize the circuit's design for efficiency and functionality.

Timing diagram

Timing diagram test is done to check whether the VHDL program and schematic diagram is valid for the given truth table and code implementation of ALU as shown in figure 3. Here, the output is implemented according to testbench file.

Device Utilization

IO (Input/Output) refers to the physical FPGA pins used for communication with external devices, tracking how many of these pins are utilized for inputs and outputs. Utilization, on the other hand, measures the internal FPGA resource consumption, such as lookup tables

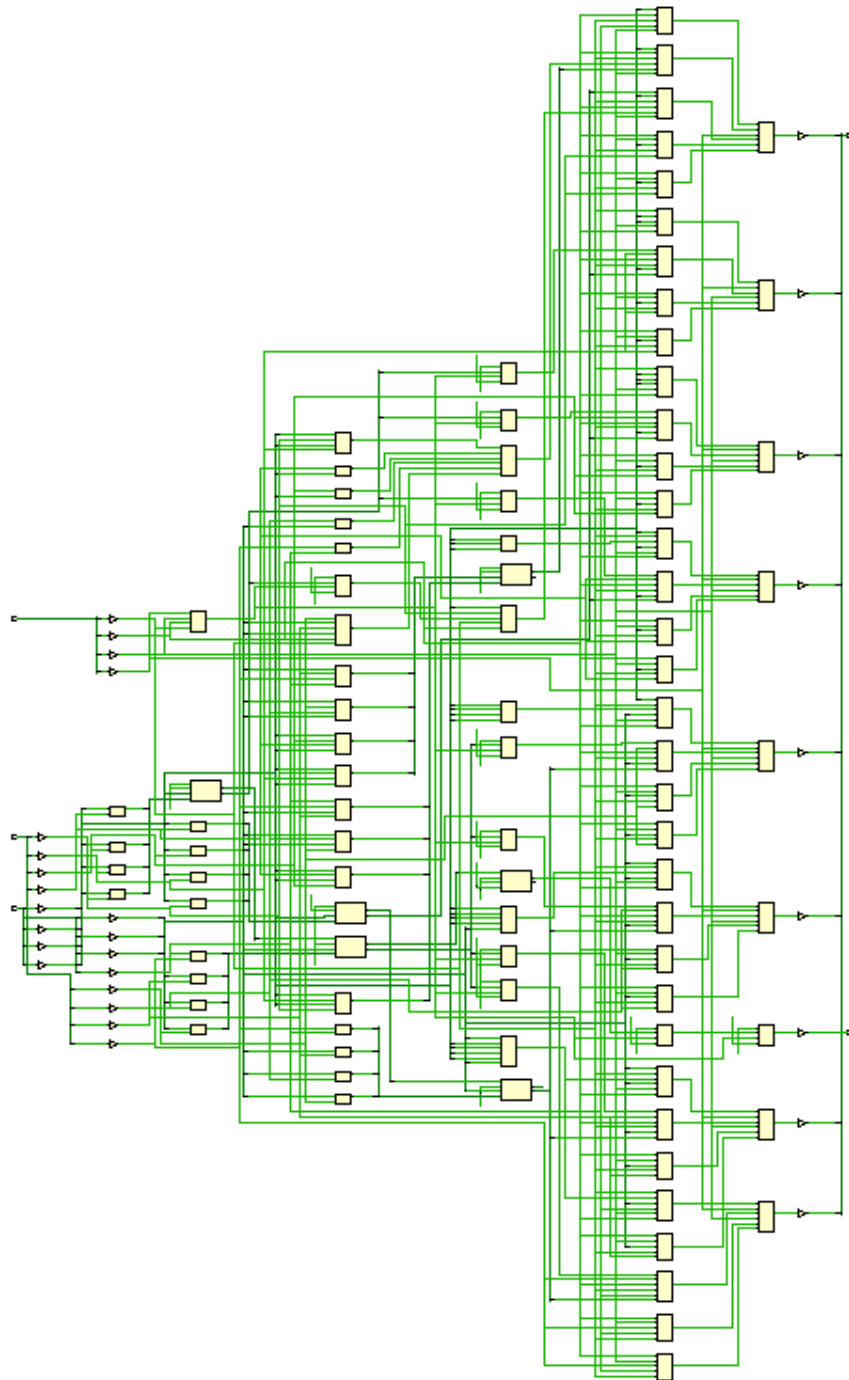


Figure 2: Synthesis Schematic Diagram of ALU

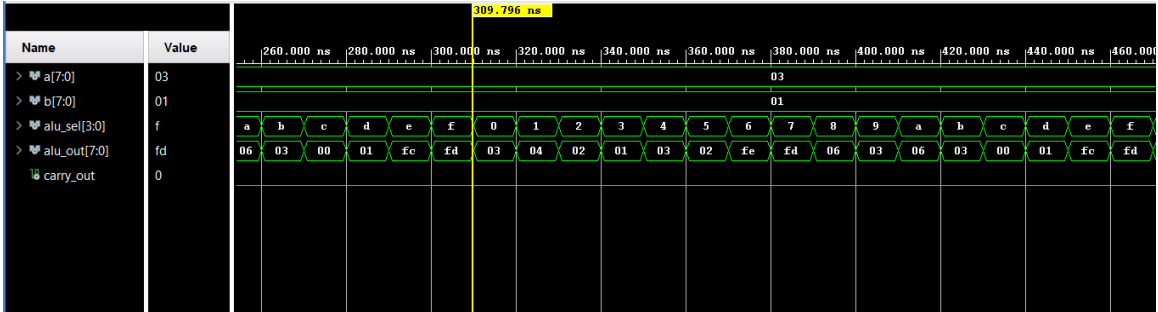


Figure 3: Timing Diagram of ALU

(LUTs), flip-flops, block RAM (BRAM), and DSP slices. Together, these concepts provide a comprehensive view of FPGA design efficiency, as shown in figure 4.

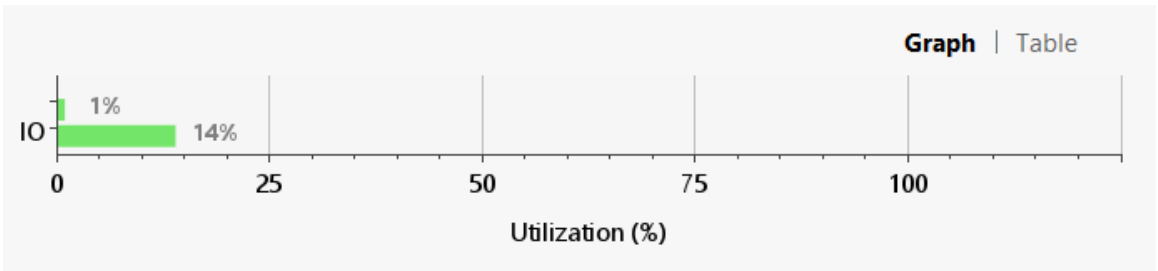


Figure 4: Device Utilization of ALU

Power Estimation

Figure 5 shows estimation of static power, dynamic power (power required when VHDL program is burned in FPGA) and other related power datas of a ALU circuit.

3 Conclusion

In this lab, we implemented the VHDL code for 8-bit ALU on the Atrix-7 FPGA (part: xc7a50tcs324-1, package: csg324) using Vivado software. Each design was synthesized, simulated, and implemented, followed by hardware testing, which confirmed their correct functionality.

The process began with simulation to verify the timing diagrams, followed by linting to check for syntax errors. Afterward, the designs were synthesized into a hardware logic circuit, and during the implementation phase, power and timing characteristics were analyzed, displayed and studied.

8 output bits were successfully obtained when 8 input bits were provided through 4 bit combinations of selection lines for which type of operation to perform.

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: **5.239 W**
Design Power Budget: **Not Specified**
Process: *typical*
Power Budget Margin: **N/A**
Junction Temperature: **50.0°C**
 Thermal Margin: 35.0°C (7.3 W)
 Ambient Temperature: 25.0 °C
 Effective θ_{JA} : 4.8°C/W
 Power supplied to off-chip devices: 0 W
 Confidence level: *Low*

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

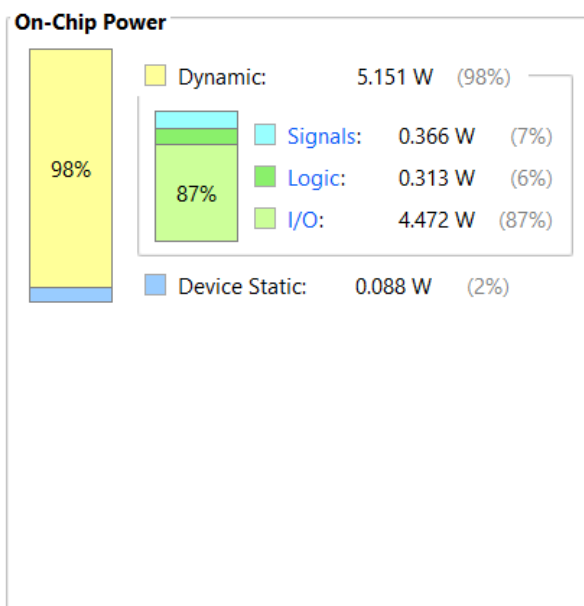


Figure 5: Power Estimation of ALU